

Is Java Alive and Kicking?

이희승

2026년 6월

네, 자바 아직 팔팔합니다!

- 6개월 릴리즈 주기
- 각종 "현대화" 프로젝트들

Project Amber	언어 개선
Project Loom	동시성 개선
Project Panama	FFI & 메모리 접근 개선
Project Leyden	기동성능 개선
Project Valhalla	Value objects

그러나

- 기술지형
- 거버넌스
- 마인드셰어

기술지형 – Single-tenancy

- 강력한 머신에 소수의 서비스를 장시간 운용
- JVM의 강력한 런타임 최적화
 - 긴 기동 시간
 - 피크 성능 도달에 시간이 필요
- 높은 throughput에 최적화된 GC 구현
 - 높은 memory footprint
- 그때는 옳았다
 - 다른 서비스와 자원을 공유하지 않았기에

기술지형 – Multitenancy

- 다수의 서비스가 가상화/컨테이너 계층을 통해 함께 실행
- 타 서비스와 자원을 공유
- 자원 상황에 따라 역동적으로 scale-in/out, migration을 반복
- JVM의 강점이 제약으로...
 - 잦은 migration vs. 긴 기동시간 및 반복적 JIT
 - Noisy neighbors vs. Memory footprint

기술지형 – Rich runtime

- 풍부한 기능을 갖는 강력한 런타임
 - Reflection
 - Class loading
 - Instrumentation
 - Adaptive optimization
 - JVMTI/HotSwap
 - ...

기술지형 – Shift-down to OS

- OS 레벨 observability의 발전
 - perf
 - eBPF
 - ...
- JVM의 introspection 기능은 여전히 강력하지만...
 - JFR / async-profiler
 - Multitenant / Polyglot 환경의 도래
 - 모든 언어에 통하는 OS 레벨의 도구가 많은 부분을 대체

기술지형 – Shift-down to build time

- 커져가는 의문
 - “빌드 타임에 할 수 있는 일들도 런타임에 하고 있었던 것은 아닐까?”
- 언어 개발 접근성
 - LLVM 생태계 (공유 가능한 컴파일러 백엔드의 등장)
 - 고수준 언어로도 가능해진 부트스트래핑
 - 축적된 자료 (예: Crafting Interpreters)
- 개발 장비 성능
 - Build time vs. Runtime 균형점 이동을 가속

기술지형 – Shift-down to build time

- Profile-guided optimization은?
 - 비대한 런타임을 정당화할 정도는...
- GraalVM은?
 - CE vs. Oracle GraalVM 기능 차별
 - 빌드에 필요한 상당한 CPU와 RAM
 - 떨어지는 성능
 - 어려운 트러블슈팅
 - 생태계 파편화 – OpenJDK와 앞으로의 관계는?
- Leyden은?
 - 가장 현실적인 대안이자 절충안 (성능 힌트/캐시)
 - 비대한 런타임은 그대로

기술지형 – Minimal tooling

- 전통적으로 언어는 컴파일러/표준 라이브러리 제공에 집중
- 커뮤니티에 의존하는 부분이 많음
 - 코딩 스타일 강제
 - 정적 분석
 - 테스트
 - 의존성 관리
 - 빌드 시스템
 - 개발 환경 통합
 - ...

기술지형 – All-in-one tooling

- 요즘 언어는 더 많은 것을 공식적으로 제공
 - 코딩 스타일 강제
 - 정적 분석
 - 테스트
 - 의존성 관리
 - 빌드 시스템
 - 개발 환경 통합 (LSP 및 IDE plugin)
 - ...
- 개발환경의 fragmentation이 낮음
- 커뮤니티의 실험적 시도를 흡수하여 “sane defaults” 추구

바뀐 지형, 두 갈래 대응

- 기술지형의 변화
 - Multitenancy · Shift-down to OS/build-time · All-in-one tooling
- 두 갈래 대응
 - ① 자바를 고친다 – “현대화” 프로젝트
 - ② 자바를 떠난다 – JVM 위의 다른 언어 (Kotlin · Scala)
- 다른 길, 같은 벽
 - 끝내 벗지 못하는 자바 · JVM의 짐

1 자바를 고친다 – “현대화” 프로젝트

- 실제로 많은 도움이 된 좋은 시도
 - Amber · Loom · Panama · Leyden · Valhalla
- 하지만 어쩔 수 없는 “혁신가의 딜레마”
 - 시작점의 기술지형 자체가 다르다는 한계
 - Loom, Panama, Leyden이 나오기까지 들어간 고민과 노력, 시간...
- 하위 호환성이라는 저주
 - 2014년부터 시작한 Valhalla, 2026년에도 미리보기
 - 아직 다루지 못한, 어쩌면 영영 바로잡지 못할
 - Nullability, Coding style & Linter, ...

2 자바를 떠난다 – Kotlin · Scala

- 정체된 자바의 빈틈을 메우며 빠르게 성장
 - 언어 ergonomics – 간결성 · Nullability · 패턴 매칭
- 그러나 "자바가 아닌 길"의 구조적 한계
 - 런타임의 짐은 그대로
 - 같은 JVM → 긴 기동시간 · 높은 footprint · JIT warmup
 - 상호운용성 세금 – 자바 생태계에 올라탔으니 자바에 묶인다
 - Kotlin platform type (String!) – 자바 API를 부르는 순간 새는 null 안전성
 - Scala-Java 컬렉션 · implicit 경계의 마찰
 - 거버넌스 · 라이선스 짐도 그대로

2 자바를 떠난다 – Kotlin · Scala

- 점점 줄어드는 차별점
 - 쉬운 승리는 자바가 흡수
 - records · pattern matching · lambda ...
 - 비슷한 컨셉, 다른 구현 = 또 하나의 상호운용성 세금
 - "탈출"을 정당화하기엔 부족한 나머지 차별점
 - extension function · implicit ...
- 탈출구라기엔 우회로 – 독이 든 성배
 - 🏆 "자바가 싫으면 JVM 위 다른 언어로 도망치면 되잖아?"
 - 💀 런타임 · 거버넌스의 짐에 상호운용성 세금까지

게다가

- 기술지형
- **거버넌스**
- 마인드셰어

거버넌스 – 언어로 이윤을 남기려다 보니

- 하나가 아닌 JDK
- 혼란스러운 라이선싱 정책
 - "수시로 바뀌는"이라고 쓰고
 - "나날이 나빠지는"이라고 읽는다
 - 종업원 수에 비례한 구독 과금이라니...
- 부정적 신호들
 - TCK license for Apache Harmony
 - Oracle v. Google
 - javax.* 네임스페이스 인질극

거버넌스 – 있어야 할 게 없다

- 자바를 처음 접한 개발자는 도대체 어디서 무얼 해야?
- 제대로 된 공식 웹 사이트?
 - 너무 늦게 생긴 dev.java
- 공식적으로 관리되는 커뮤니티?
- 진정한 의미의 공식 빌드?
 - Java는 Oracle 꺼니까 Oracle JDK 쓰면 되나?
 - 라이선스가 복잡하니까 OpenJDK 라는 걸 써야 되나??
 - Temurin은 뭐고 Adoptium은 뭔지...???

비극의 시작:

“돌볼 공유재”가 아닌

“통제하고 수익화할 자산”으로 언어를 바라봄

게다가

- 기술지형
- 거버넌스
- **마인드셰어**

마인드셰어 – 요즘 친구들의 생각

- 2024년 베를린 MERGE 컨퍼런스에서 부스를 운영
 - 여러 스타트업의 젊은 친구들이 부스를 방문
 - 자바로 신규 프로젝트를 진행하는 비율은 10% 이내
- 더 이상 "다음에 배우고 싶은 언어"가 아닌 자바
 - JetBrains 2025 설문
 - Language Promise Index상 성장 잠재력 1-3위는 TypeScript · Rust · Go
- Haskell, Erlang/Elixir 처럼 열광적인 팬도 없다
- 그도 그럴 것이...
 - 공홈은 도대체 어디?
 - 라이선스는 뭐가 뭔지...
 - npm, cargo 같은 거 없나? Maven/Gradle 등 더 배울 게 있다고?

마인드셰어 – 당장은 관참을 지 몰라도

- 큰 기업에게 사랑받은 자바
 - 그들에게도 어린 시절이 있었는데...
 - 여전히 꽤 높은 점유율
- “다음” 유니콘도 자바를 선택할까?
 - 트위터가 고래 띄우던 시절의 기술지형과
 - 클로드가 529 에러 내는 지금의 기술지형은 다르다

지금까지의 정리

- 기술의 지형이 많이 바뀌었다
 - Multitenancy
 - Shift-down to OS and build-time
 - All-in-one tooling
- 자바는 바뀐 지형에 맞춰 혁신중이고, 나름 성공했다
 - Amber, Loom, Leyden, Panama
- 하지만 자바는...
 - 어쩔 수 없는 세월의 짐을 지고 있다
 - 다른 언어들이 치고 올라오기 전에 서둘러 혁신을 시작했어야 했다
 - JVM 기반 대안 언어의 노력은 한계가 있다
 - 해결 불가능에 가까운 거버넌스 이슈가 있다

자바의 마인드셰어 현실:

2023년 대한민국 합계출산율?

자바는 반등에 성공할까?

- 개인적으로는 비관적
- 한 번 놓친 버스를 따라잡기란 쉽지 않다
 - Laravel이 흥해도 PHP의 옛 영광이 회복되지 않았듯...
 - 기술적 우수성과 마인드세어 회복은 별개
 - 한 번 떠난 신규 프로젝트 · 젊은 세대가 기술이 좋아졌다고 돌아올까?
- 당장은 힘들겠지만 기회는 있다
 - Ruby on Rails를 만난 Ruby의 성장을 기억하자
 - 한 언어의 미래는 그 언어를 사용하고 개발하는 이들의 손에...

변화라는 이름의 상수

- 기술지형은 끊임없이 변화해 옴
- 엔지니어는 상황에 적응하고 가장 적합한 기술을 채용
 - 과거 - 조금 적합하지 않아도 익숙한 기술을 선택
 - 현재 - AI의 등장 = 조금 익숙치 않아도 최적의 기술을 선택
- 자바가 다른 기술 대비 더 적합한 경우가 감소했을 뿐
 - 한때 자바가 다른 기술들을 밀어내었듯...

우리는 어떡하면 좋을까?

- 기술지형의 변화에 적응하자
- 다양한 선택지가 있음을 인정하자
 - 상황에 맞는 기술을 선택하고 조합하자
 - 웹 프론트엔드에 자바를 쓰지 않듯,
모든 백엔드에서 자바를 쓸 필요는 없다
 - Rust로 작성한 모듈을 Panama로 호출하면?
 - 익숙하지 않겠지만 AI를 활용하면 큰 도움이 된다
- 행동으로 변화를 이끌어내자
 - 💪💪 팔팔한 활동 → 팔팔한 자바
 - ※ 자바 "팔"을 쓰는 것은 팔팔한 게 아님에 유의 🚫

Q&A

감사합니다